

Class 04: Props, Layouts, Pages, and Styling

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Props



- Props เป็น parameters ของ component ระบุ property ของ components
  - กำหนด props เข้าไปเป็น property ของ component ได้ เช่น props `style`

```
<View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
  <Text>Hello, world!</Text>
</View>
```

  - เช่น button มี props เช่น `title` เป็นต้น

```
<button title='Hello'>Click Me</button>
```

  - ใน JSX ค่าของ props สามารถเป็นค่าตัวแปร หรือเป็นผลลัพธ์ของ Java Script ได้ โดยวางไว้ใน `{}`

```
<button title={myTitle}>Click Me</button>
```

Props

- **Props**
  - Developer มักจะสร้าง custom components โดยกำหนด props ให้กับ component ที่สร้างขึ้นได้
  - สร้าง Functional component ที่รับ props หากใช้ TypeScript ต้องระบุ Data Type ของ props ด้วย

การระบุ Data Type สำหรับ Props ใน Type Script

- นิยาม Type ของ Props โดยการใช้ keyword `interface` เพื่อสร้าง structural subtyping ระบุ shape ของ value สำหรับ Object (contract สำหรับ object) โดยใช้ syntax ต่อไปนี้

Greeting.tsx

```tsx
interface Props {
    name: string;
    lastname: string;
}

export default function Greeting (user:Props) {
    return (
        <p>Hello {user.name} {user.lastname}.</p>
    );
}
```

นำ Greeting ไปใช้งาน

```tsx
<Greeting name='Peter' lastname='Foster'/>
```

Hello Peter Foster.

Props

- **Props**
  - หากใช้ **object destructuring** ใน function signature ก็สามารถเข้าถึง attribute ได้โดยตรง
  - ตัวอย่างการใช้งานกับ interface

Greeting.tsx

```tsx
interface Props {
    name: string;
    lastname: string;
}

export default function Greeting({ name, lastname }:Props) {
    return (
        <p>Hello {name} {lastname}.</p>
    );
}
```

นำ Greeting ไปใช้งาน

```tsx
<Greeting name='Peter' lastname='Foster'/>
```

Hello Peter Foster.

Props

การระบุ Data Type สำหรับ Props ใน Type Script

- นิยาม Type ของ Props โดยการใช้ keyword type เพื่อสร้าง alias ให้กับ type หรือกลุ่มของ type

มีเครื่องหมาย =

Greeting.tsx

```tsx
type Props = {
    name: string;
    lastname: string;
}

export default function Greeting (user:Props) {
    return (
        <p>Hello {user.name} {user.lastname}.</p>
    );
}
```

นำ Greeting ไปใช้งาน

```tsx
<Greeting name='Peter' lastname='Foster'/>
```

Hello Peter Foster.

Props



- หากใช้ **object destructuring** ใน function signature ก็สามารถเข้าถึง attribute ได้โดยตรง
- ตัวอย่างการใช้งานกับ type

Greeting.tsx

```tsx
type Props = {
    name: string;
    lastname: string;
}

export default function Greeting({ name, lastname }:Props) {
    return (
        <p>Hello {name} {lastname}.</p>
    );
}
```

นำ Greeting ไปใช้งาน

```tsx
<Greeting name='Peter' lastname='Foster'/>
```

Hello Peter Foster.

Props



- **Props**
  - ระบุ types parameter ให้กับ Functional Components ด้วย Syntax ต่อไปนี้

Greeting.tsx

```tsx
export default function Greeting (user: { name: string; lastname: string; } ){
    return (
        <p>Hello {user.name} {user.lastname}.</p>
    );
}
```

  - props ได้รับการระบุมาจาก parent component

```tsx
<Greeting name='Peter' lastname='Foster'/>
```

นำ Greeting ไปใช้งาน

Hello Peter Foster.

Props



- หากใช้ **object destructuring** ใน function signature ก็สามารถเข้าถึง attribute ได้โดยตรง
- ตัวอย่างการใช้งานกับ function parameter

Greeting.tsx

```tsx
export default function Greeting({ name, lastname }:{ name:string, lastname:string }) {
    return (
        <p>Hello {name} {lastname}.</p>
    );
}
```

```tsx
<Greeting name='Peter' lastname='Foster'/>
```

นำ Greeting ไปใช้งาน

Hello Peter Foster.

Props



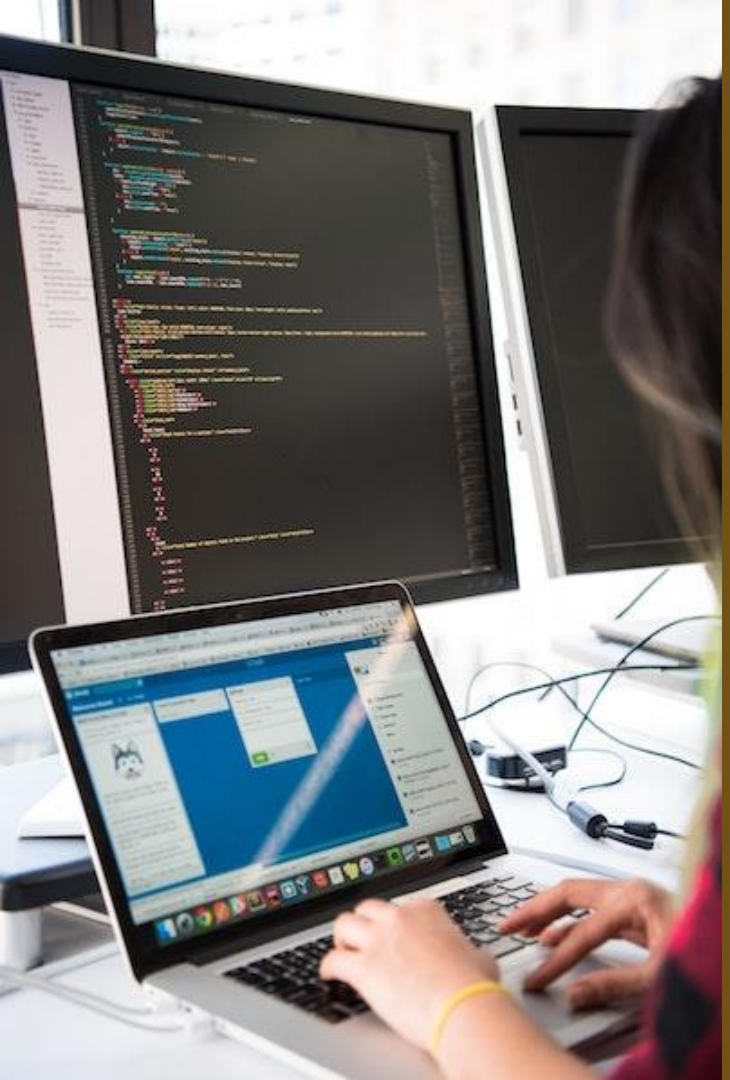
- การส่ง props จาก Parent component ไปยัง Child Component สามารถทำได้หลายวิธี
  - ส่งแบบแยก value

```
<Greeting name='Peter' lastname='Foster'/>
```

  - ส่งโดยใช้ spread operator (...) เพื่อส่งข้อมูลแยกไปยัง props

```
export default function Home() {
  const data = {
    name: "Peter",
    lastname: "Foster"
  }
  return (
    <Greeting {...data}/>
  )
}
```

```
<Greeting {...{name: "Peter",lastname: "Foster"}}/>
```

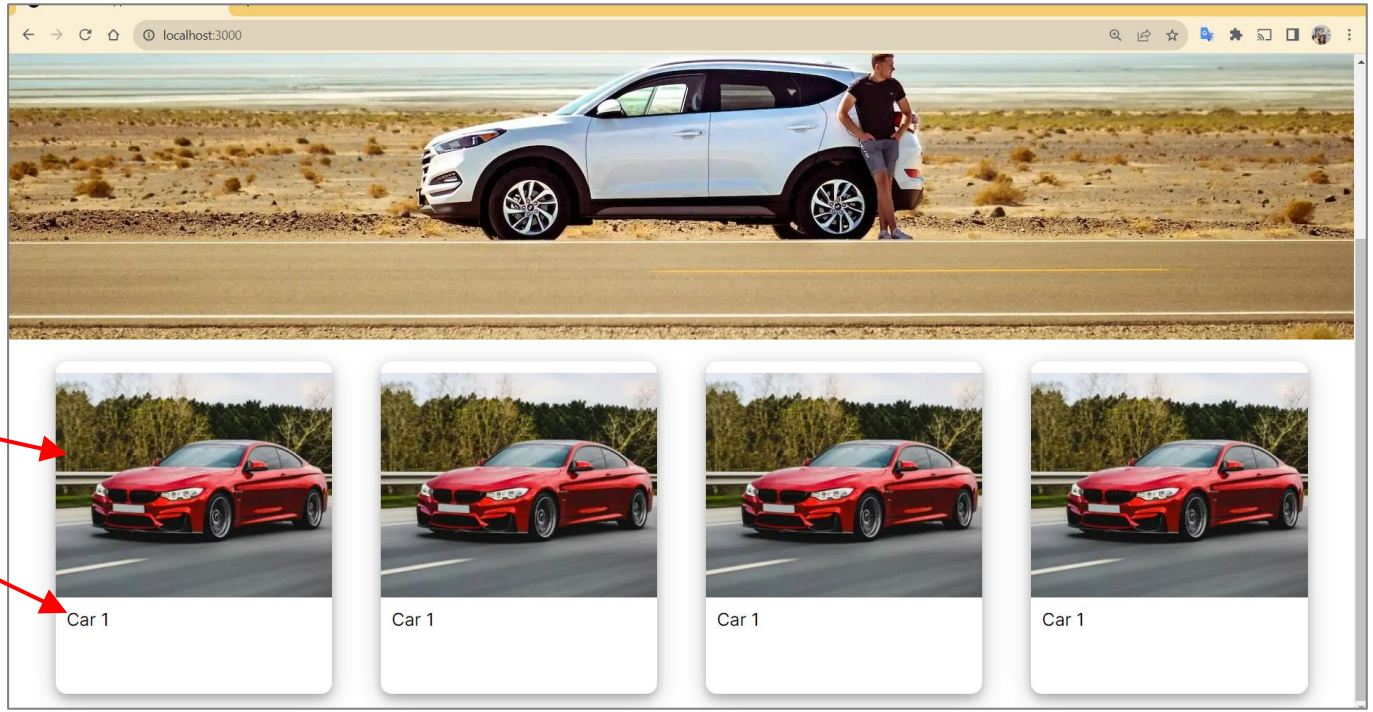


Workshop 4.1 : Component Props

Continue with Car Rental Workshop

- Workshop นี้ เป็น workshop ต่อจาก Week 03 ตัวอย่าง car-rental-app

- เราจะให้ component `ProductCard` รับ Props จาก Page ซึ่งเป็น parent component ได้แก่
  - รูปภาพรถ
  - ชื่อรถ



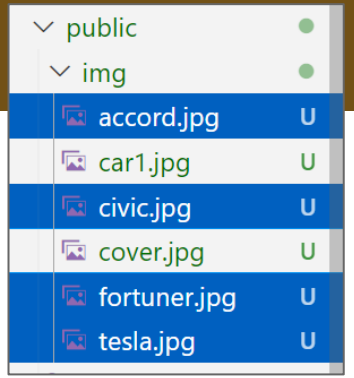
Props for Product Card



- หน้า components/ProductCard.tsx ปรับให้ component นี้
  - รับ props ได้แก่ carName และ imgSrc
  - นำ carName ไปแสดงเป็นข้อความชื่อรุ่นรถ และ imgSrc ไปเป็น props src ให้แก่ Image component

```
export default function ProductCard({ carName, imgSrc }:{ carName:string, imgSrc:string }) {
    return (
        <div className={styles.card}>
            <div className={styles.cardimg}>
                <Image src={imgSrc}
                alt='Product Picture'
                fill={true}
                objectFit='contain'
                />
            </div>
            <div className={styles.cardtext}>{carName}</div>
        </div>
    );
}
```

Props for Product Card



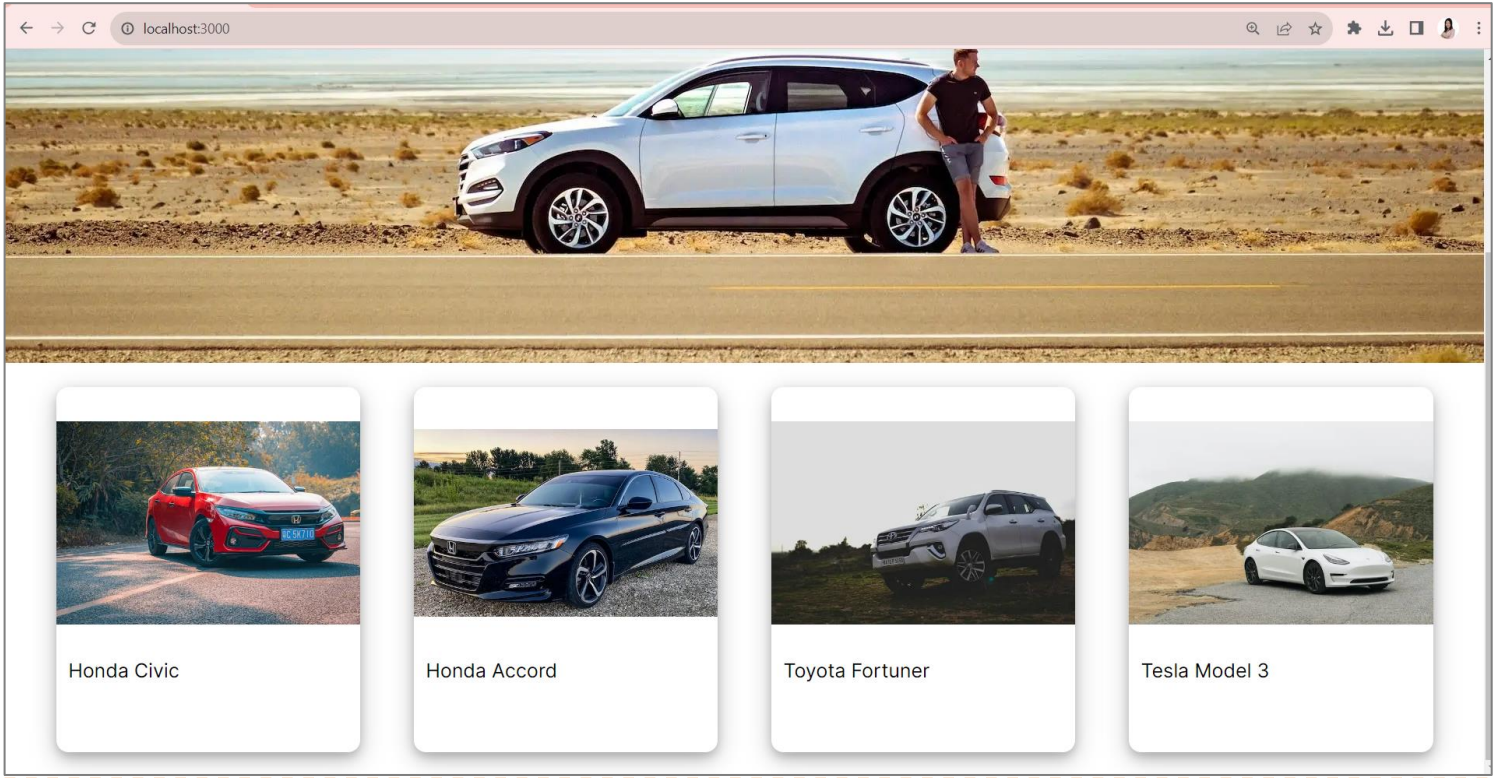
- นำภาพรถที่ต้องการนำมาแสดง มาวางไว้ใน `public/img`

- หน้า `page.tsx` ปรับให้ page นี้
  - ส่ง props ได้แก่ `carName` และ `imgSrc` ให้แก่ component `ProductCard`

```
<main>
  <Banner/>
  <div style={{margin:"20px", display:"flex", flexDirection:"row",
  flexWrap:"wrap", justifyContent:"space-around", alignContent:"space-around"}}>
    <ProductCard carName='Honda Civic' imgSrc='/img/civic.jpg'/>
    <ProductCard carName='Honda Accord' imgSrc='/img/accord.jpg'/>
    <ProductCard carName='Toyota Fortuner' imgSrc='/img/fortuner.jpg'/>
    <ProductCard carName='Tesla Model 3' imgSrc='/img/tesla.jpg'/>
  </div>
</main>
```

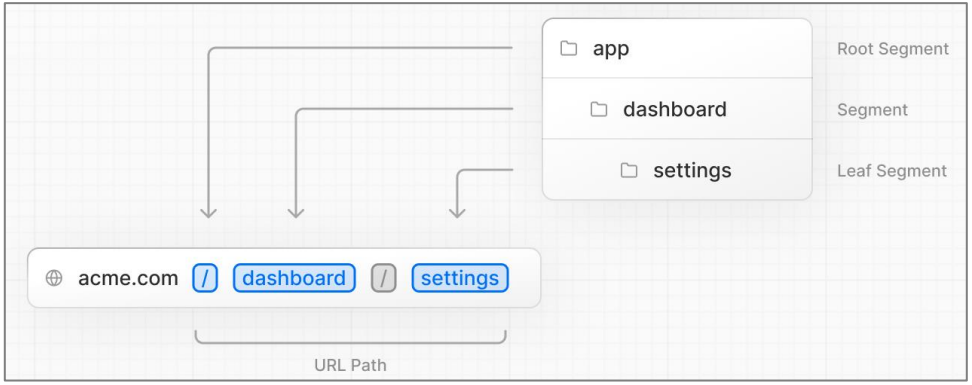
Props for Product Card

- Run application เพื่อสังเกตุผลลัพธ์

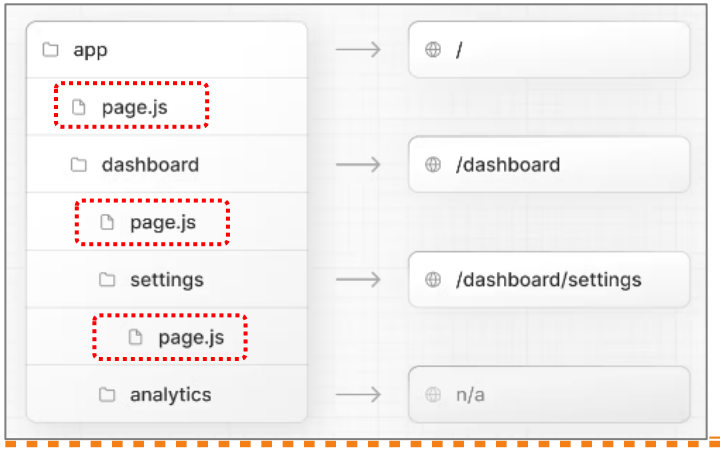


Next.js Basic: Pages & Layouts

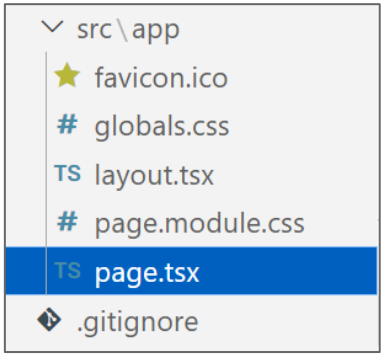
- Next.js ใช้ file-system based router โดยใช้ folder ในการระบุ route ต่าง ๆ
  - แต่ละ folder นำเสนอ route segment ที่ map เข้ากับ URL segment
  - สร้าง nested route ผ่าน nest folders



- ไฟล์ `page.js` หรือ `page.jsx` หรือ `page.tsx` เป็นไฟล์พิเศษที่ทำให้ route segments นั้น ๆ มองเห็นเป็นสาธารณะ



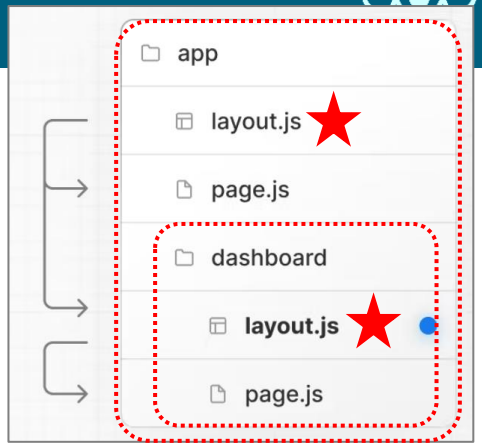
Next.js Basic: Pages & Layouts



- **Page**
  - page เป็น UI ที่ unique สำหรับ route
  - ระบุ page โดยการ export component จากไฟล์ `page.js` หรือ `page.jsx` หรือ `page.tsx`

- **Layout**
  - Layout เป็น UI ที่ share ระหว่างหลาย page
  - ในระหว่างการ navigate นั้น layout จัดเก็บ state และไม่ได้รับการ re-render
  - ระบุ layout โดย default ด้วยการ export React component จากไฟล์ `layout.js` หรือ `layout.jsx` หรือ `layout.tsx`
  - layout อาจซ้อนกันได้ เรียกว่าเป็น **nested layout**

Pages & Layouts

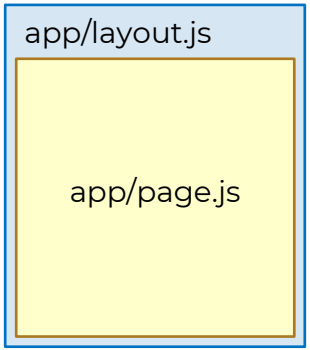


- **Nested Layouts**
  - Next.js รองรับ layout หลายอันซ้อนกัน โดย default แล้ว layout จะซ้อนกันตามลำดับชั้นของไฟล์

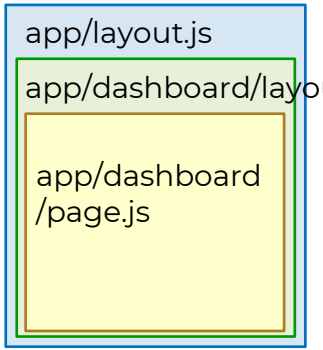
จากภาพตัวอย่าง

- Layout ใน `app/layout.js` จะเป็น layout ใหญ่
- Layout ใน `app/dashboard/layout.js` จะถูก activate ก็ต่อเมื่อมีการ navigate ไปยังหน้า `domain.com/dashboad`
- Layout ใน `app/layout.js` หุ้ม layout ใน `app/dashboard/layout.js` อีกที

`domain.com`



`domain.com/dashboad`



Next.js Basic: Pages & Layouts



- **Layout**
  - สร้าง layout โดยการ export React component ในไฟล์ `layout.js` หรือ `layout.jsx` หรือ `layout.tsx`
  - React component ที่เป็น layout ต้องรับ `children` prop ที่จะถูก populate ระหว่างการ render โดย Child page หรือ Child layout (ถ้ามี)

```tsx
export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body className={inter.className}>{children}</body>
    </html>
  )
}
```

  - มักพบรูปแบบของ functional component ที่เป็น layout เป็น

```tsx
export default function layout_name ({ children } : { children: React.ReactNode }) {

}
```

Next.js Basic: Pages & Layouts



- **Root Layout**
  - คือ Layout ระดับบนสุด เป็น layout จำเป็นที่จะแชร์ข้ามหน้าทั้งหมดใน application
  - Root layout ต้องบรรจุ Tag `<html> ... </html>` และ `<body> ... </body>`

app/layout.tsx

```tsx
export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body className={inter.className}>{children}</body>
    </html>
  )
}
```

Populate children component มาใส่ใน body

Links



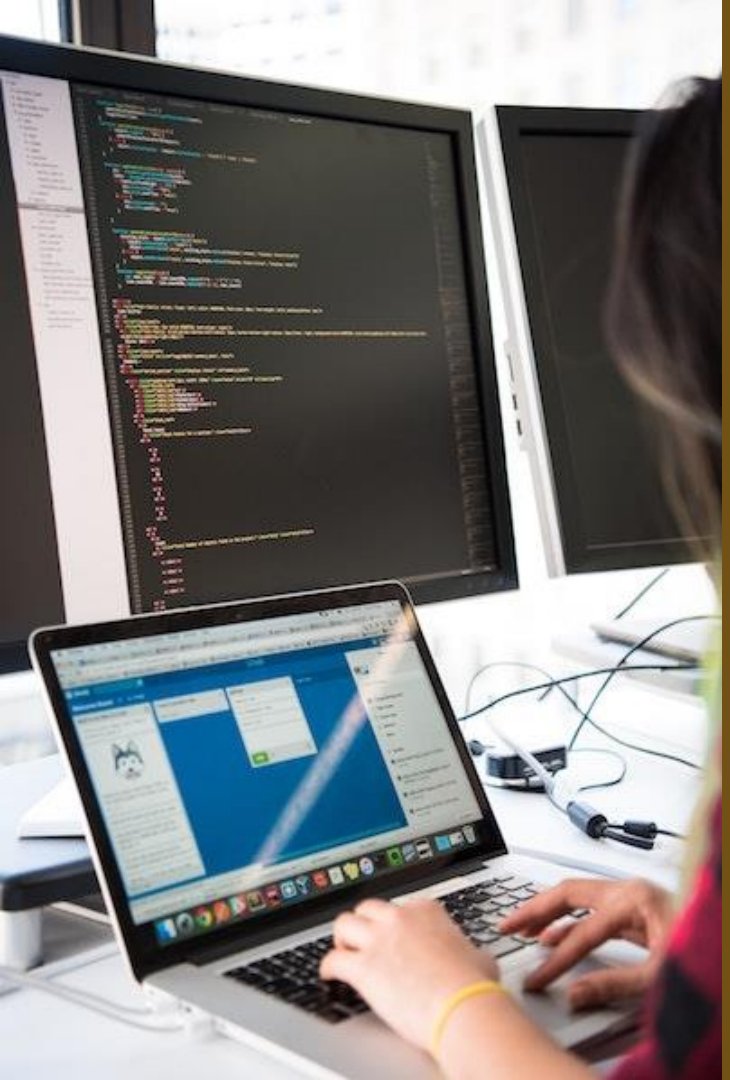
- `<Link>` Component เป็น built-in component ใน Next.js ที่ต่อยอดจาก HTML <a> tag เพื่อใช้สำหรับการทำ prefetching และ navigation ไปยัง route ต่าง ๆ บนฝั่ง client-side
  - ใช้งานโดยการ import `Link` Component จาก `next/link`

```
import Link from 'next/link'
```

  - ระบุปลายทางผ่าน `href` prop ตัวอย่างเช่น

```
<Link href="/dashboard">Dashboard</Link>
```

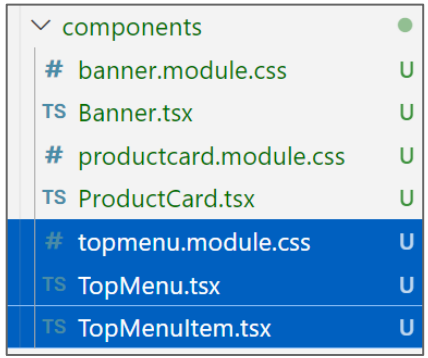
  - ตัวอย่าง props อื่น ๆ
    - `scroll` ถ้า `true` จะ scroll ไปยังด้านบนสุดของ route ใหม่ ถ้า `false` จะรักษา scroll position เดิม
    - `prefetch` ถ้า `true` จะ pre-fetch หน้าของ route ใหม่ เมื่อตัว link ถูก load มาแสดงใน Viewport
    - `replace` ถ้า `true` จะเป็นการแทนที่สถานะปัจจุบันใน history ของ Browser ถ้า `false` จะเพิ่ม URL ใหม่เข้าไปใน history ของ Browser

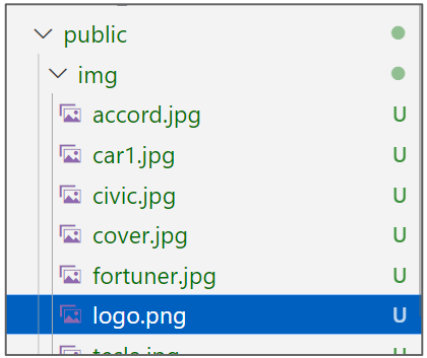


Workshop 4.2 : Page & Layout

Menu in Layout for all pages

- ในโฟลเดอร์ components สร้าง
  - ไฟล์ TopMenu.tsx สำหรับ component ที่จะเป็น Top Menu ด้านบน และ TopMenuItem.tsx สำหรับ component ที่จะเป็น Menu Item ต่าง ๆ
  - ไฟล์ topmenu.module.css สำหรับเก็บ style ของ Menu
- นำภาพ logo logo.png มาวางไว้ใน public/img





- ไฟล์ topmenu.module.css สร้าง style สำหรับ div ที่จะเป็นตัว Menu และสำหรับภาพ logo

```css
.menucontainer {
    height: 50px;
    background-color: white;
    position: fixed;
    top: 0px;
    left: 0px;
    right: 0px;
    z-index: 30;
    border-top: 1px solid lightgrey;
    border-bottom: 1px solid lightgrey;
    display: flex;
    flex-direction: row;
}

.logoimg {
    height: 100%;
    width: auto;
}
```

Menu in Layout for all pages



- ไฟล์ components /TopMenu.tsx สร้าง Functional component สำหรับ Top Menu ด้านบนของ layout

```tsx
import styles from './topmenu.module.css'
import Image from 'next/image';

export default function TopMenu () {
    return (
        <div className={styles.menucontainer}>
            <Image src={'/img/logo.png'} className={styles.logoimg}
            alt='logo' width={0} height={0} sizes="100vh"/>
        </div>
    );
}
```

- ไฟล์ layout.tsx import และวาง component TopMenu ไว้ด้านบนของ layout

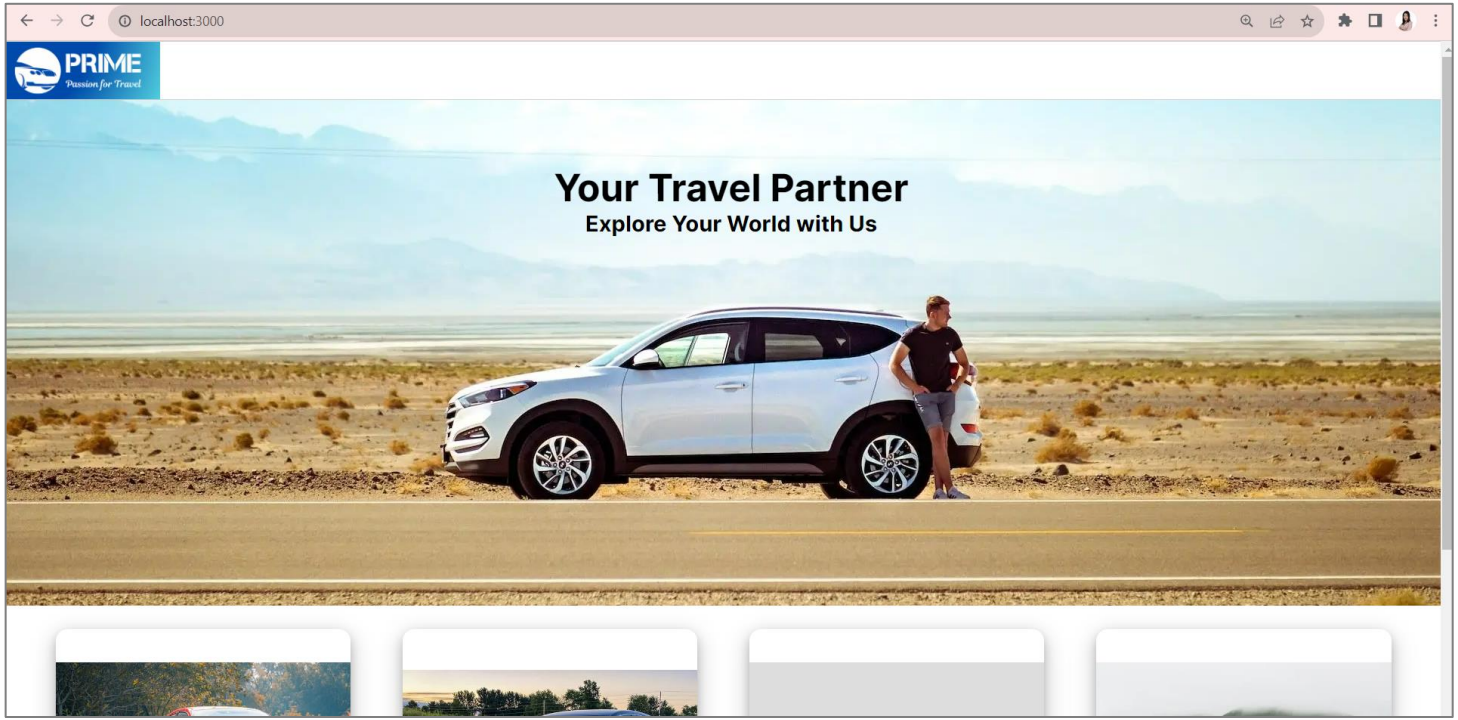
src > app > layout.tsx > RootLayout

```tsx
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
import TopMenu from '@/components/TopMenu'
```

```tsx
return (
  <html lang="en" className={`${geis
    <body>
      <TopMenu />
      {children}
    </body>
  </html>
);
```

Menu in Layout for all pages

- จะได้ Top Menu พร้อม logo อยู่ด้านบน



Menu in Layout for all pages



- ไฟล์ `topmenu.module.css` สร้าง style สำหรับ Menu Item
- ไฟล์ `TopMenuItem.tsx` สร้าง Functional Component สำหรับ Menu Item โดยรับ props ได้แก่ `title` ข้อความของ menu item และ `pageRef` ปลายทางที่จะ link ไป

```css
.itemcontainer {
    width: 120px;
    text-align: center;
    margin-top: auto;
    margin-bottom: auto;
    font-family:Verdana, Geneva, Tahoma, sans-serif;
    font-size: 10pt;
    color: grey;
}
```

```tsx
import styles from './topmenu.module.css'
import Link from 'next/link';

export default function TopMenuItem ({ title, pageRef }:{ title:string, pageRef:string }) {
    return (
        <Link className={styles.itemcontainer} href={pageRef}>
        {title}
        </Link>
    );
}
```

Menu in Layout for all pages



- ไฟล์ `TopMenu.tsx` วาง `TopMenuItem` component ลงไปต่อจาก logo รวม 2 item

```tsx
import Image from 'next/image';
import TopMenuItem from './TopMenuItem';

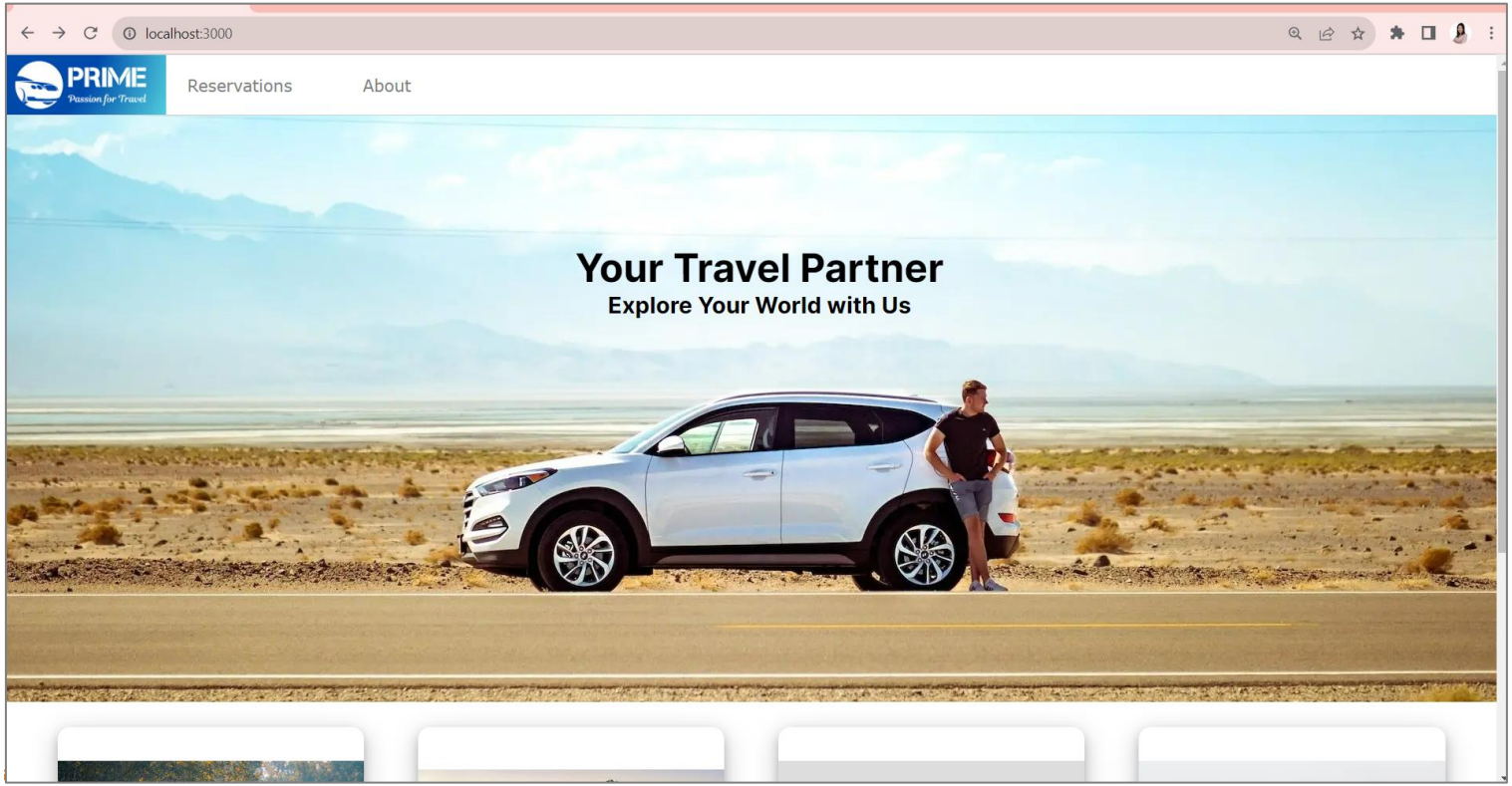
export default function TopMenu () {
    return (
        <div className={styles.menucontainer}>
            <Image src={'/img/logo.png'} className={styles.logoimg}
            alt='logo' width={0} height={0} sizes="100vh"/>
            <TopMenuItem title='Reservations' pageRef='/reservations'/>
            <TopMenuItem title='About' pageRef='/about'/>
        </div>
    );
}
```

- Top Menu จะวางอยู่ด้านบนของทุกหน้า สูง 50px จึงต้องปรับให้ทุกหน้าเว้นระยะด้านบน 50px
  - ไฟล์ `global.css` เพิ่ม `margin-top` ให้ tag `body`

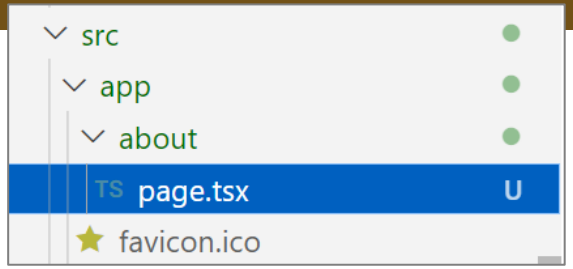
```css
body {
  min-height: 100%;
  display: flex;
  flex-direction: column;
  color: var(--foreground);
  background: var(--background);
  font-family: Arial, Helvetica, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  margin-top: 50px;
}
```

Menu in Layout for all pages

- จะได้ Top Menu พร้อม Menu Item โดยเนื้อหาทั้งหน้า เว้นระยะลงมาเท่ากับความสูงของ Top Menu



Menu in Layout for all pages



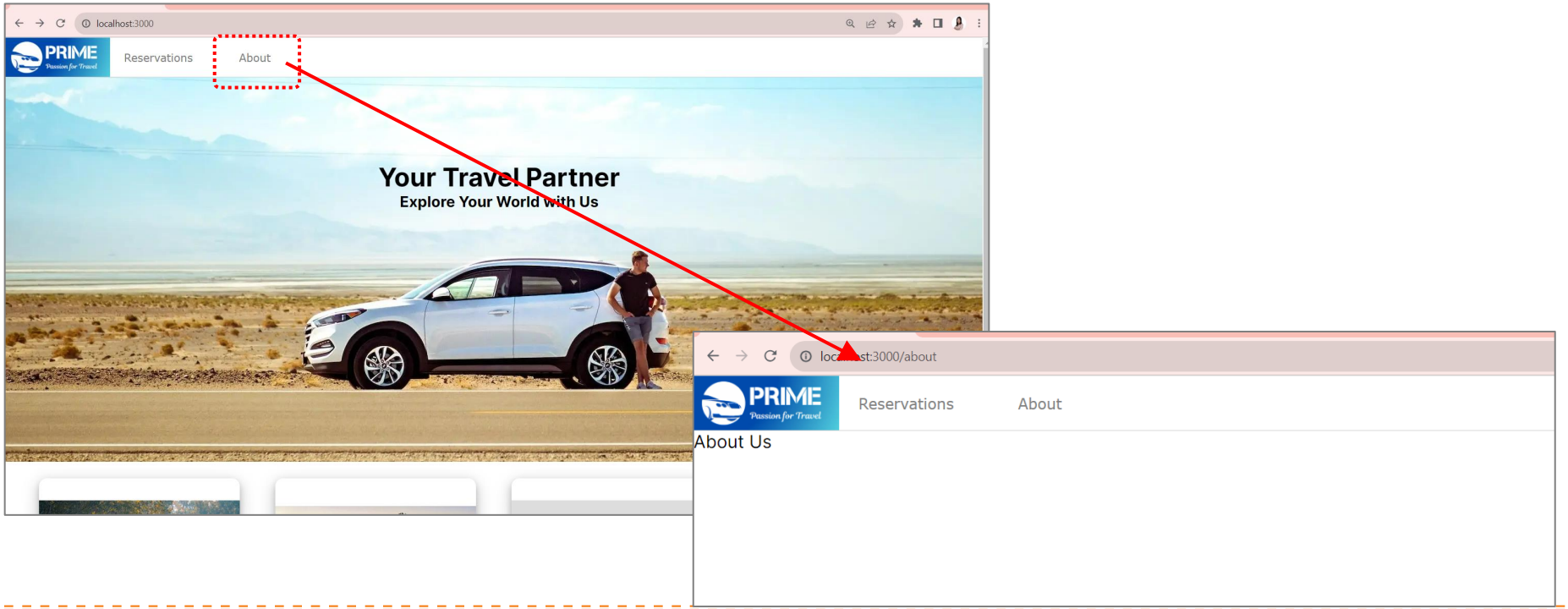
- สร้าง Route about โดย
  - สร้างโฟลเดอร์ about ใน src/app
  - สร้าง page.tsx ในโฟลเดอร์ about

- ลองสร้าง page component ในไฟล์ src/app/about/page.tsx

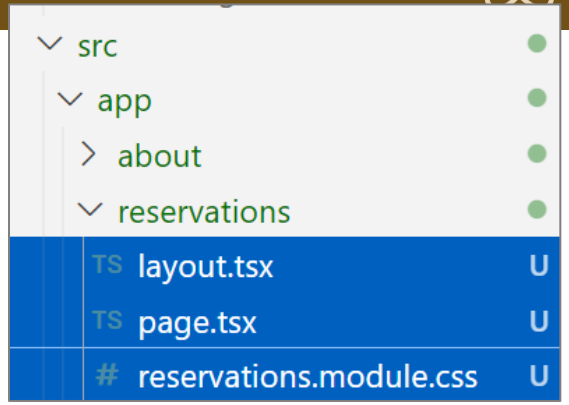
```tsx
export default function About() {
    return (
        <main>
            <div>About Us</div>
        </main>
    );
}
```

Menu in Layout for all pages

- ลอง navigate จาก http://localhost:3000 ไป http://localhost:3000/about
  - ทั้งสอง route ใช้ layout ร่วมกัน และมี Top Menu อยู่ด้านบน



Submenu as nested layout



- สร้าง Route `reservations` โดย
  - สร้างโฟลเดอร์ `reservations` ใน `src/app`
  - สร้าง `layout.tsx` ในโฟลเดอร์ `reservations`
  - สร้าง `page.tsx` ในโฟลเดอร์ `reservations`
  - สร้าง `reservations.module.css` ในโฟลเดอร์ `reservations`

- ลองสร้าง layout ในไฟล์ `src/app/reservations/layout.tsx`

```tsx
export default function ReservationLayout({ children }: { children: React.ReactNode }) {
    return (
        <div>
            {children}
        </div>
    )
}
```

Submenu as nested layout

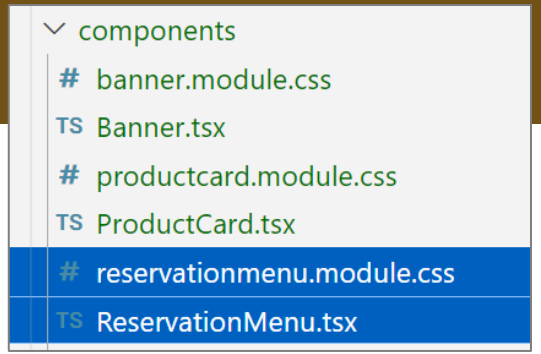
- ลองสร้าง page component ในไฟล์ `src/app/reservations/page.tsx`

```tsx
export default function Reservations() {
    return (
        <main>
            <div>New Reservation</div>
        </main>
    );
}
```

- ลอง navigate จาก http://localhost:3000 ไป http://localhost:3000/reservations



Submenu as nested layout



- ในโฟลเดอร์ components
  - สร้าง ReservationMenu.tsx
  - สร้าง reservationmenu.module.css

- ใน reservationmenu.module.css สร้าง Style ให้กับ Submenu Container
- ใน ReservationMenu.tsx สร้าง Submenu component

```css
div.submenu {
    width: 250px;
    background-color: white;
    border-radius: 10px;
    border: 1px solid silver;
    padding: 10px;
    margin: 5px;
}
```

```tsx
import styles from './reservationmenu.module.css'
export default function ReservationMenu() {
    return (
        <div className={styles.submenu}>
            Sub-Menu Here
        </div>
    );
}
```

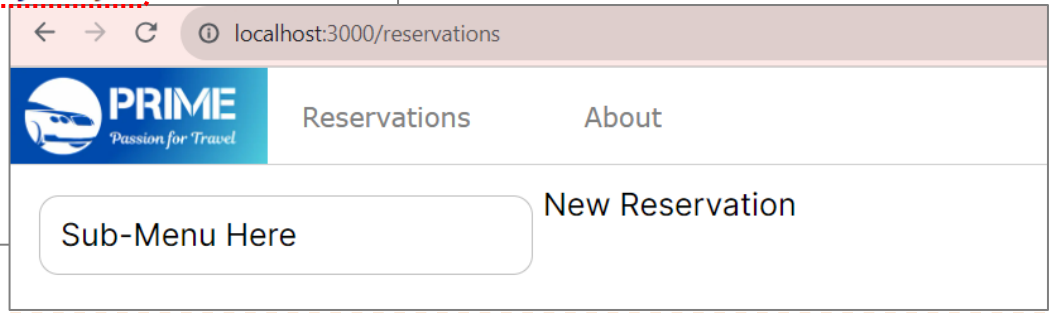
Submenu as nested layout

- จัดวางหน้า Layout เพื่อเพิ่ม sub-menu เข้าไปบน layout
  - ใน `reservations/reservations.module.css` สร้าง Style ให้กับ layout ของ route reservations
  - ใน `reservations/layout.css` เพิ่ม `ReservationMenu` component ลงไปใน layout

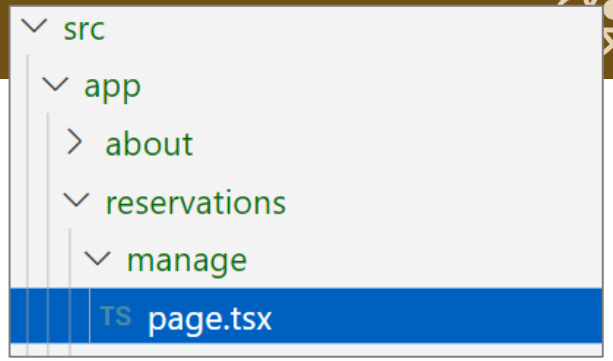
```css
.sectionlayout {
    padding: 10px;
    display: flex;
    flex-direction: row;
}
```

```jsx
import styles from './reservations.module.css'
import ReservationMenu from "@/components/ReservationMenu"

export default function ReservationLayout({ children }: { chi
    return (
        <div className={styles.sectionlayout}>
          <ReservationMenu/>
            {children}
        </div>
    )
}
```



Submenu as nested layout



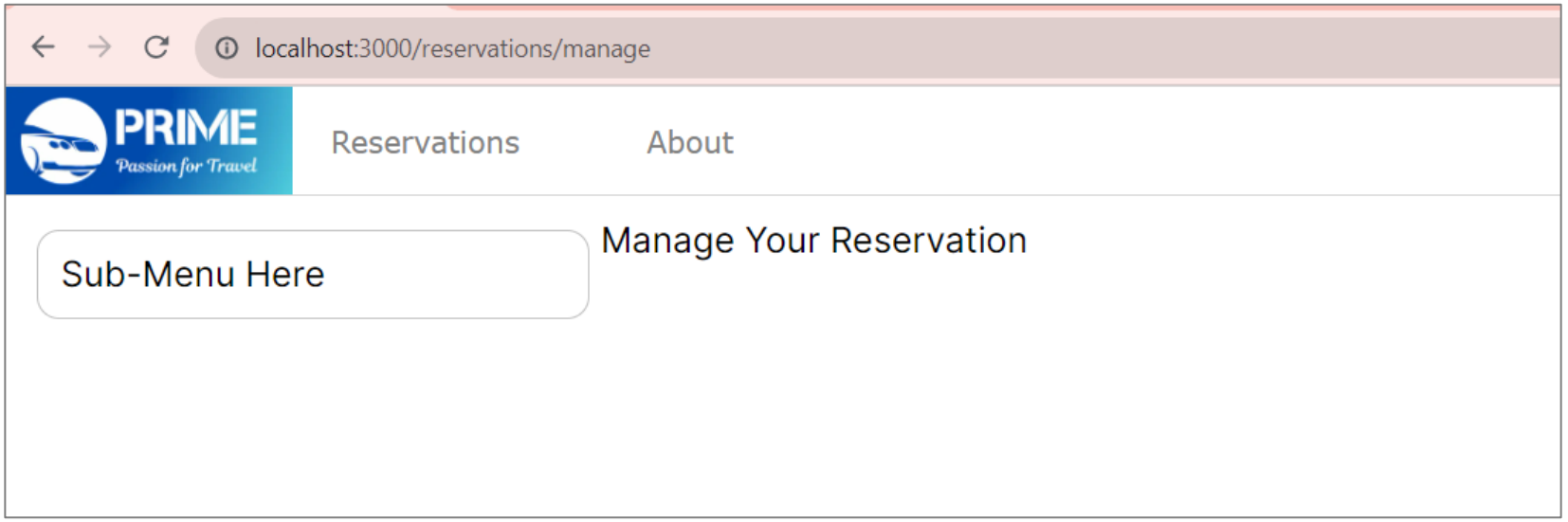
- สร้าง Route `reservations/manage` โดย
  - สร้างโฟลเดอร์ `manage` ใน `src/app/reservations`
  - สร้าง `page.tsx` ในโฟลเดอร์ `manage`

- ลองสร้าง page ในไฟล์ `src/app/reservations/manage/page.tsx`

```tsx
export default function ManageReservations() {
    return (
        <main>
            <div>Manage Your Reservation</div>
        </main>
    );
}
```

Menu in Layout for all pages

- ลอง navigate ไป http://localhost:3000/reservations/manage
  - route `reservations/manage` ใช้ layout ร่วมกันกับ route `reservations`



Tailwind CSS



- Next.js ทำงานได้ดีกับ Tailwind CSS ซึ่งเป็น CSS framework ที่มาพร้อม CSS class พร้อมใช้งาน เพื่อช่วยในการ style website
  - เพิ่มเติม https://tailwindcss.com/

- ติดตั้ง Tailwind CSS ใน Next.js
  - ดูจาก https://tailwindcss.com/docs/installation/framework-guides/nextjs

  - ใช้คำสั่ง `npm install` ติดตั้ง Tailwind CSS และ dependencies ที่จำเป็น

```
npm install tailwindcss @tailwindcss/postcss postcss
```

Tailwind CSS

- ติดตั้ง Tailwind CSS ใน Next.js
  - สร้างไฟล์ `postcss.config.mjs` ที่ project root เพิ่ม @tailwindcss/postcss plugin เข้าไปใน PostCSS configuration

postcss.config.mjs

```
1  const config = {
2    plugins: {
3      "@tailwindcss/postcss": {},
4    },
5  };
6  export default config;
```

  - เพิ่ม การ import Tailwind CSS เข้าไปใน `global.css`

globals.css

```
1  @import "tailwindcss";
2  :root {
3    --background: #ffffff;
```

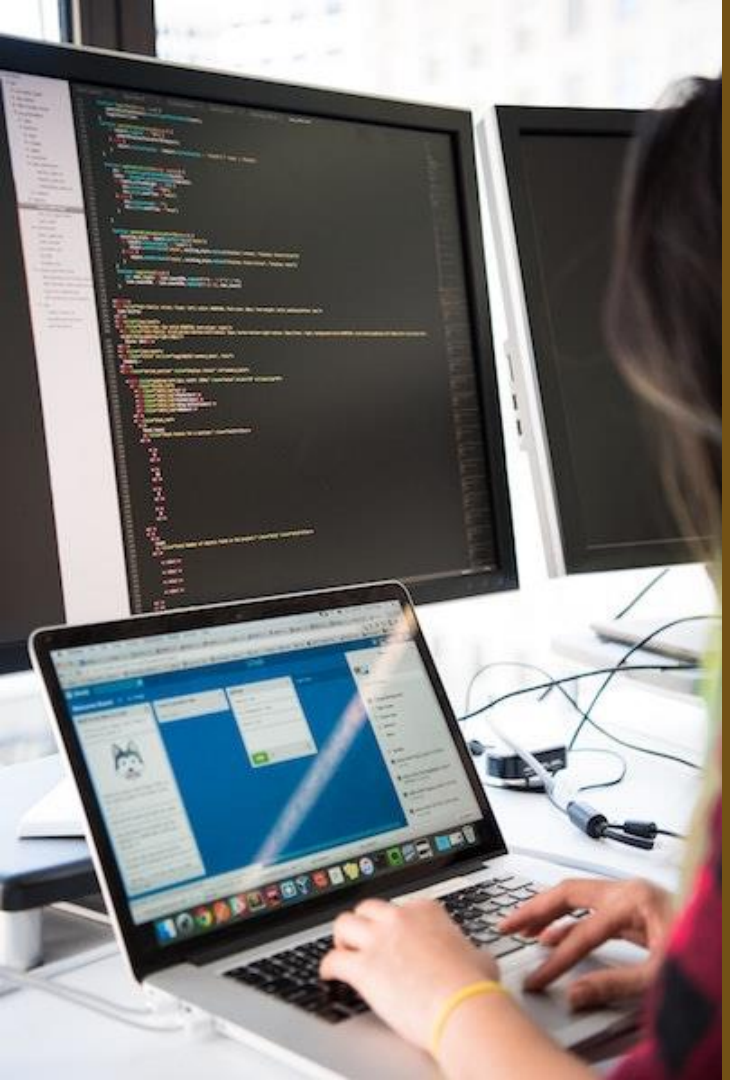
Tailwind CSS



- สามารถใช้งาน Functionality ต่าง ๆ ของ Tailwind CSS ได้

  ดู Tailwind CSS Documentation

  - https://tailwindcss.com/docs
  - ขนาดฟอนต์ รูปแบบฟอนต์ น้ำหนักฟอนต์
    - https://tailwindcss.com/docs/font-size
    - https://tailwindcss.com/docs/font-weight
  - ขนาด ตำแหน่ง เงา
    - https://tailwindcss.com/docs/height
    - https://tailwindcss.com/docs/width
    - https://tailwindcss.com/docs/padding
    - https://tailwindcss.com/docs/position
    - https://tailwindcss.com/docs/object-fit
    - https://tailwindcss.com/docs/box-shadow
    - https://tailwindcss.com/docs/border-radius



Workshop 4.3 :
Start Using Tailwind CSS

Installing Tailwind CSS



- ติดตั้ง Tailwind CSS จาก console โดย
  - Terminate ตัว server โดยการกด `Ctrl + c`
  - ใช้คำสั่ง `npm install` ติดตั้ง Tailwind CSS และ dependencies ที่จำเป็น

```
PS D:\SWDevPrac2\ReactWeb\car-rental-app> npm install tailwindcss @tailwindcss/postcss postcss
```

  - สร้างไฟล์ `postcss.config.mjs` ที่ project root เพิ่ม `@tailwindcss/postcss` plugin

postcss.config.mjs

```
1  const config = {
2      plugins: {
3        "@tailwindcss/postcss": {},
4      },
5    };
6  export default config;
```

Installing Tailwind CSS

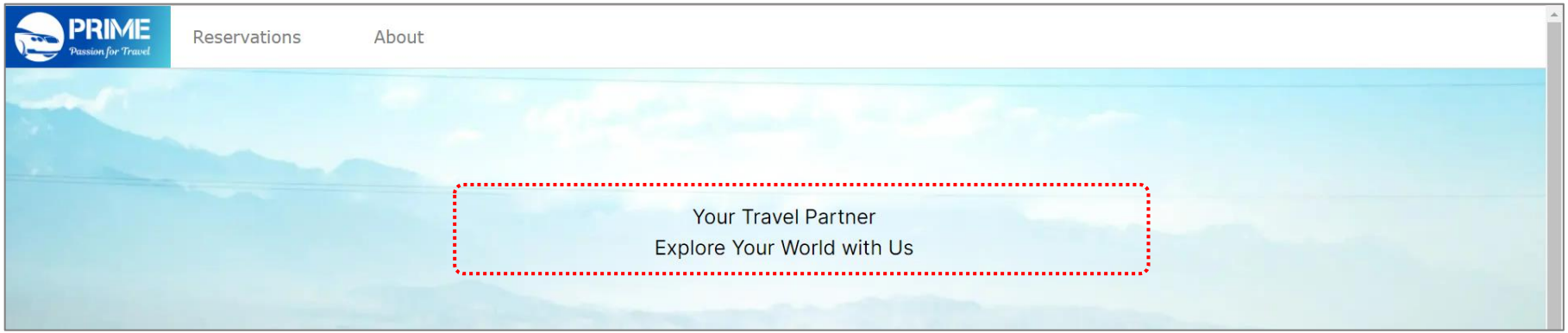


- ใน `global.css` เพิ่ม การ import Tailwind CSS

```
# globals.css
src > app > # globals.css > :root
1  @import "tailwindcss";
2  :root {
3    --background: #ffffff;
```

Using Tailwind CSS

- จะพบว่า Styling ของหน้าเว็บเปลี่ยนไป



Using Tailwind CSS

- ใน components/Banner.tsx ปรับแต่ง Text Style โดยใช้ class จาก Tailwind CSS

```
<div className={styles.bannerText}>
    <h1 className='text-4xl font-medium'>Your Travel Partner</h1>
    <h3 className='text-xl font-serif'>Explore Your World with Us</h3>
</div>
```



Using Tailwind CSS



- ลองใช้งาน Tailwind CSS ในการ style ProductCard แทน CSS ธรรมดา

```
return (
    <div className="w-1/5 h-[300px] rounded-lg shadow-lg">
        <div className="w-full h-[70%] relative rounded-t-lg">
        <Image src={imgSrc}
            alt='Product Picture'
            fill={true}
            className='object-cover rounded-t-lg'
            />
        </div>
        <div className="w-full h-[30%] p-[10px]">{carName}</div>
    </div>
);
```

- สำหรับ Next.js 16.3 template ลบการ reset padding เป็น 0 ใน global.css

```
* {
  box-sizing: border-box;
  padding: 0;
  margin: 0;
}
```

Using Tailwind CSS



- Tailwind CSS ในการ style `ProductCard` แทน CSS ธรรมดา

